Taller 2 – Restricciones, Adversarios y Drones
Reflexión sobre el trabajo realizado (Punto 4)

Nombres: Diego Munévar, Joshua Ross, Martin Leschhorn, Juan Sebastián Ávila Códigos: - 202322221 - 202326898 - 202320954 - 202320465

15 de marzo de 2026

Introducción

En este documento se presentará el análisis correspondiente del taller, esto incluye los análisis de complejidad temporal y espacial y una comparación cualitativa/cuantitativa del desempeño de los algoritmos basada en pruebas realizadas sobre los distintos layouts.

4(a) Análisis de los algoritmos CSP

Punto 1 – Backtracking search
Contexto del problema: Es el backtracking clásico sin ninguna optimización el cual asigna rutas a los drones al probar diferentes combinaciones de rutas y retrocede cuando encuentra una violación de restricciones de batería, capacidad o tiempo.
Complejidad en espacio (Big-O)

$$\boldsymbol{O(n)}$$

Donde:

- n: es el número de variables del CSP, el cual es un dron que debe recibir una ruta o destino.

Esto es debido a que el algoritmo mantiene únicamente la asignación parcial de rutas a los drones y la pila de recursión del algoritmo, y en lo peores casos el algoritmo debe recorrer una profundidad de n niveles, uno por cada dron que se asigna por lo que el espacio crece proporcionalmente al número de drones en la misión.

Complejidad en temporalidad (Big-O)

$$\boldsymbol{O(d^n)}$$

Donde:

- n: es el número de variables del CSP, el cual es un dron que debe recibir una ruta o destino.
- d: es el número máximo de rutas o destinos posibles de cada dron.

El tiempo de demora está fuertemente influenciado en probar todas las combinaciones posibles de asignación de rutas, lo que genera un árbol de búsqueda más grande de tamaño $\boldsymbol{d^n}$. En el peor de los casos cuando hay muchas restricciones este valor crecerá proporcionalmente al número de estas ya que cada intento de asignación el código verifica si la asignación cumple las restricciones.

Punto 2 – Backtracking foward checking
Contexto del problema: El foward checking permite al algoritmo eliminar de forma anticipada rutas que se vuelven invalidas teniendo en cuenta las restricciones. Esto permite detectar inconsistencias de manera más eficiente antes de explorar más niveles del árbol de búsqueda.

Complejidad en espacio (Big-O)

$$\boldsymbol{O(d * n)}$$

Donde:

- n: es el número de variables del CSP, el cual es un dron que debe recibir una ruta o destino.
- d: es el número máximo de rutas o destinos posibles de cada dron.

El espacio que ocupa ocurre principalmente porque se debe almacenar los dominios de cada variable y además registrar temporalmente los valores eliminados durante el foward checking para poder restaurarlos cuando ocurre backtracking. En el peor de los casos se deben almacenar n dominios de tamaño d.

Complejidad en temporalidad (Big-O)

$$\boldsymbol{O(n * d^{n+1})}$$

Donde:

- n: es el número de variables del CSP, el cual es un dron que debe recibir una ruta o destino.
- d: es el número máximo de rutas o destinos posibles de cada dron.

El tiempo se debe a que se sigue explorando un árbol de búsqueda potencial de tamaño $\boldsymbol{d^n}$, similar al backtracking básico. El código lo que realiza es revisar los dominios de los drones vecinos y eliminar valores inconsistentes, lo que puede implicar revisar hasta n dominios de tamaño d, más el costo adicional en cada nodo del árbol (de ahí el +1)

¿Cómo se reduce el espacio de búsqueda comparado con una búsqueda exhaustiva simple? Compare cuantitativamente el número de asignaciones intentadas con ejemplos extraídos de sus experimentos.

Layout	Estados BT	Estados FC	Backtracks BT	Backtracks FC	Checks BT	Checks FC	Podas FC
twin_bases	4	3	1	1	5	7	3
triple_fleet	4	4	0	0	4	11	1
jungle_outpost	4	4	0	0	4	11	1
tight_battery	3	3	0	0	3	4	1
heavy_cargo	4	4	0	0	4	9	2
storm_corridor	3	3	0	0	3	4	1
round_trip	3	3	0	0	2	3	0
tight_windows	4	4	0	0	4	8	1
big_fleet	8	7	2	2	21	40	13
battery_edge	2	2	0	0	1	1	0
detour_required	2	2	0	0	1	1	0
hero_drone	4	4	0	0	5	8	2
weight_distribution	4	4	0	0	6	11	3

fog_valley	2	2	0	0	2	2	0
mountain_bypass	2	2	0	0	1	1	0
staggered_time	4	4	0	0	4	9	2
maze_csp	3	3	0	0	2	4	0
cross_mission	3	3	0	0	3	5	1
tight_overload	5	5	0	0	6	13	2
single_path	4	4	0	0	3	6	0
corner_challenge	3	3	0	0	3	4	1
storm_shortcut	3	3	0	0	3	4	1

Backtracking con Forward Checking mantiene el mismo número de soluciones, pero realiza poda anticipada de dominios, reduciendo ligeramente el número de estados explorados en algunos layouts. Sin embargo, debido al tamaño reducido de las instancias, el beneficio en términos de backtracking es limitado y el número de verificaciones de restricciones aumenta debido al costo adicional del chequeo hacia adelante.

Punto 3 – Backtracking con AC-3

Contexto del problema: Combina el backtracking con AC-3, que elimina valores de dominio que no cumplen consistencia entre variables conectadas por restricciones.

Complejidad en espacio (Big-O)

$$\boldsymbol{O(d * n + c)}$$

Donde:

- n: es el número de variables del CSP, el cual es un dron que debe recibir una ruta o destino.
- d: es el número máximo de rutas o destinos posibles de cada dron.
- c: es el número de restricciones entre drones o rutas.

Al algoritmo necesitar almacenar los dominios de todas las variables y además mantener los pares de variables que se revisan en el algoritmo AC-3. Lo que hace en el peor de los casos que se sume la cantidad de restricciones que se deban verificar.

Complejidad en temporalidad (Big-O)

$$\boldsymbol{O(c * d^{n+3})}$$

Donde:

- n: es el número de variables del CSP, el cual es un dron que debe recibir una ruta o destino.
- d: es el número máximo de rutas o destinos posibles de cada dron.
- c: es el número de restricciones entre drones o rutas.

El tiempo se ve influenciado de manera importante por el algoritmo AC-3 que compara los valores de los dominios de pares de variables para asegurar consistencia de arcos. En el peor de los casos cada verificación puede requerir comparar todos los valores de dos dominios lo que genera un costo de (+3) por restricción. También al ejecutarse dentro del proceso de backtracking que hace que se exploren $\boldsymbol{d^n}$ combinaciones, lo que hace que el costo crezca multiplicativamente.

¿Cómo se reduce el espacio de búsqueda comparado con una búsqueda exhaustiva simple? Compare cuantitativamente el número de asignaciones intentadas con ejemplos extraídos de sus experimentos.

Layout	Estados BT	Estados AC3	Backtracks BT	Backtracks AC3	Checks BT	Checks AC3	Podas AC3	AC3 Calls	Arc Revisions
twin_bases	4	4	1	1	5	12	2	3	2
triple_fleet	4	4	0	0	4	15	0	3	3
jungle_outpost	4	4	0	0	4	14	0	3	3
tight_battery	3	3	0	0	3	6	0	2	1
heavy_cargo	4	4	0	0	4	12	0	3	3
storm_corridor	3	3	0	0	3	5	1	2	1
round_trip	3	3	0	0	2	3	0	2	1
tight_windows	4	4	0	0	4	11	1	3	4
big_fleet	8	8	2	2	21	121	4	7	22
battery_edge	2	2	0	0	1	1	0	1	0
detour_required	2	2	0	0	1	1	0	1	0
hero_drone	4	4	0	0	5	12	2	3	5
weight_distribution	4	4	0	0	6	22	1	3	4
fog_valley	2	2	0	0	2	2	0	1	0
mountain_bypass	2	2	0	0	1	1	0	1	0
staggered_time	4	4	0	0	4	12	0	3	3
maze_csp	3	3	0	0	2	4	0	2	1
cross_mission	3	3	0	0	3	7	1	2	1
tight_overload	5	5	0	0	6	22	0	4	6
single_path	4	4	0	0	3	6	0	3	3
corner_challenge	3	3	0	0	3	6	0	2	1
storm_shortcut	3	3	0	0	3	5	1	2	1

La incorporación de AC-3 permite realizar poda anticipada mediante consistencia de arcos. Sin embargo, en los layouts evaluados el número de estados explorados y backtracks permanece igual al backtracking estándar. Debido al tamaño reducido de los problemas, el costo adicional de ejecutar AC-3 se refleja principalmente en un aumento significativo en el número de verificaciones de restricciones.

Punto 4 – Backtracking forward checking con heurísticas MRV y LCV

Contexto del problema: La heurística utilizada inicialmente en foward checking cambia por las heurísticas de MRV y LCV para elegir variable más restrictiva y los valores que generen menos conflictos entre sí.

Complejidad en espacio (Big-O)

$$\boldsymbol{O(n * d)}$$

Donde:

- n: es el número de variables del CSP, el cual es un dron que debe recibir una ruta o destino.
- d: es el número máximo de rutas o destinos posibles de cada dron.

El espacio necesitado depende que se mantengan los dominios de todas las variables y la asignación parcial de rutas. Además, durante la ejecución se evalúan temporalmente los valores del dominio al aplicar las heurísticas, lo que implica también almacenar información sobre posibles conflictos.

Complejidad en temporalidad (Big-O)

$$\boldsymbol{O(n * d^{n+1})}$$

Donde:

- n: es el número de variables del CSP, el cual es un dron que debe recibir una ruta o destino.
- d: es el número máximo de rutas o destinos posibles de cada dron.

En él, pero caso el algoritmo aún puede explorar un árbol de búsqueda de tamaño $\boldsymbol{d^n}$, donde cada nodo se ejecutan heurística de MRV para revisar las variables no asignadas para seleccionar la más restringida y LCV para calcular el número de conflictos para cada valor del dominio. Estas operaciones requieren evaluar múltiples variables y valores agregando el factor n y d para cada paso de la búsqueda.

¿Cómo se reduce el espacio de búsqueda comparado con una búsqueda exhaustiva simple? Compare cuantitativamente el número de asignaciones intentadas con ejemplos extraídos de sus experimentos.

Layout	Estados BT	Estados MRV+LCV	Backtracks BT	Backtracks MRV+LCV	Checks BT	Checks MRV+LCV	Podas MRV+LCV
twin_bases	4	3	1	1	5	7	3
triple_fleet	4	4	0	0	4	12	1
jungle_outpost	4	4	0	0	4	12	1
tight_battery	3	3	0	0	3	4	1
heavy_cargo	4	4	0	0	4	9	1
storm_corridor	3	3	0	0	3	4	1
round_trip	3	3	0	0	2	3	0
tight_windows	4	4	0	0	4	9	1
big_fleet	8	7	2	2	21	40	13
battery_edge	2	2	0	0	1	1	0
detour_required	2	2	0	0	1	1	0
hero_drone	4	5	0	2	5	15	5
weight_distribution	4	4	0	0	6	12	3
fog_valley	2	2	0	0	2	2	0
mountain_bypass	2	2	0	0	1	1	0
staggered_time	4	4	0	0	4	9	2
maze_csp	3	3	0	0	2	4	0
cross_mission	3	3	0	0	3	5	1
tight_overload	5	5	0	0	6	16	2
single_path	4	4	0	0	3	6	0
corner_challenge	3	3	0	0	3	4	1
storm_shortcut	3	3	0	0	3	4	1

El uso de las heurísticas MRV y LCV permite reducir el número de estados explorados en algunos layouts al seleccionar variables más restrictivas y valores menos conflictivos. Sin embargo, debido al tamaño reducido de los problemas evaluados, el impacto en la reducción de backtracking es limitado. Además, el número de verificaciones de restricciones aumenta debido al costo adicional del cálculo de las heurísticas.

4(b) Análisis de los algoritmos de búsqueda adversaria

- Complejidad en tiempo y espacio de Minimax en función de la profundidad d y el factor de ramificación b.

La complejidad temporal de Minimax depende del factor de ramificación b y de la profundidad de búsqueda d. En el peor caso, el algoritmo debe explorar todos los nodos del árbol hasta la profundidad especificada, por lo que su complejidad temporal es $O(b^d)$. Esto se debe a que en cada nivel del árbol se generan aproximadamente b posibles acciones y el algoritmo explora recursivamente cada una de ellas.

En cuanto a la complejidad espacia, Minimax utiliza una búsqueda en profundidad, por lo que necesita almacenar únicamente la rama actual del árbol y las llamadas recursivas activas. Por esta razón, la complejidad espacial suele expresarse como **O(bd)** o aproximadamente **O(d)** considerando principalmente la pila de recursión.

- **¿Por qué Minimax puede ser pesimista en entornos con cazadores no óptimos?**

Minimax asume que los oponentes siempre juegan de manera óptima y seleccionaran la acción que minimiza la utilidad del agente MAX. En este problema, esto implica asumir que los cazadores siempre tomaran la decisión más perjudicial posible para el dron.

Sin embargo, en escenarios reales o simulaciones donde los cazadores no siempre actúan de manera perfectamente racional o pueden tener componentes aleatorios, este supuesto puede resultar demasiado pesimista. Como consecuencia, el dron puede evitar rutas que en la práctica serian viables o seguras, ya que el algoritmo evalúa los estados considerando siempre el peor caso posible.

- **Describa y justifique su función de evaluación: ¿qué características del estado usa, qué pesos asignó y por qué?**

La función de evaluación diseñada considera varias características del estado para estimar la utilidad de posiciones no terminales. Entre las variables utilizadas se encuentran:

- "el puntaje actual del estado,"
- "el número de entregas pendientes,"
- "la distancia desde el dron hasta la entrega más cercana,"
- "la distancia desde los cazadores hasta el dron."

Se asigno una penalización significativa al número de entrega pendientes con el fin de incentivar el progreso hacia el objetivo principal del problema. Adicionalmente, se recompensa la cercanía a los puntos de entrega para favorecer ruta que acerquen al dron a completar sus objetivos.

También se considera la distancia a los cazadores para evitar estados peligrosos donde el dron puede ser capturado fácilmente. De esta manera, la función de evaluación busca equilibrar dos factores principales: el progreso hacia las entregas y la seguridad frente a los cazadores.

- **Compare las tasas de victoria de los agentes en diversos layouts.**

Algorithm	Layout	Games	Wins	Losses	WinRate	AvgScore
Expectimax	tiny_hunt	10	10	0	1.00	492.0
Expectimax	small_hunt	10	9	1	0.90	836.4
Expectimax	open_field	10	10	0	1.00	1473.6
Expectimax	drone_escape	10	10	0	1.00	993.0
Expectimax	supply_run	10	10	0	1.00	993.0
Expectimax	swamp_run	10	10	0	1.00	1451.1
Expectimax	jungle_chase	10	7	3	0.70	439.0
Expectimax	contested_territory	10	9	1	0.90	840.3
Expectimax	maze_hunt	10	8	2	0.80	658.9
Expectimax	mountain_pass	10	10	0	1.00	950.8
Expectimax	wide_open	10	10	0	1.00	1481.4
Expectimax	bottleneck	10	10	0	1.00	989.0
Expectimax	gauntlet	10	2	8	0.20	-619.4
Expectimax	triple_threat	10	5	4	0.50	1119.5
Expectimax	scattered_hunters	10	10	0	1.00	1935.7
Expectimax	terrain_trap	10	9	1	0.90	840.3
Expectimax	storm_tunnel	10	10	0	1.00	963.0
Expectimax	arena	10	7	2	0.70	908.4
Expectimax	hunter_cage	10	10	0	1.00	971.4
Expectimax	tunnel_escape	10	10	0	1.00	486.0
Expectimax	mountain_overfly	10	9	1	0.90	806.4
Expectimax	hunter_swarm	10	5	0	0.50	641.7
Expectimax	pacman_maze	10	1	9	0.10	497.4
Expectimax	desperate_run	10	8	0	0.80	755.8
AlphaBeta	tiny_hunt	10	10	0	1.00	485.6
AlphaBeta	small_hunt	10	10	0	1.00	987.4
AlphaBeta	open_field	10	10	0	1.00	1469.0
AlphaBeta	drone_escape	10	10	0	1.00	992.0
AlphaBeta	supply_run	10	10	0	1.00	993.0
AlphaBeta	swamp_run	10	10	0	1.00	1448.2
AlphaBeta	jungle_chase	10	7	3	0.70	490.4
AlphaBeta	contested_territory	10	9	1	0.90	839.7
AlphaBeta	maze_hunt	10	8	2	0.80	620.5
AlphaBeta	mountain_pass	10	10	0	1.00	977.6
AlphaBeta	wide_open	10	10	0	1.00	1471.1
AlphaBeta	bottleneck	10	9	1	0.90	788.9
AlphaBeta	gauntlet	10	3	7	0.30	-415.6
AlphaBeta	triple_threat	10	4	5	0.40	1059.7
AlphaBeta	scattered_hunters	10	6	4	0.60	952.3
AlphaBeta	terrain_trap	10	9	0	0.90	889.1
AlphaBeta	storm_tunnel	10	9	0	0.90	894.8

AlphaBeta	arena	10	4	0	0.40	980.4
AlphaBeta	hunter_cage	10	9	1	0.90	781.5
AlphaBeta	tunnel_escape	10	10	0	1.00	487.0
AlphaBeta	mountain_overfly	10	10	0	1.00	959.2
AlphaBeta	hunter_swarm	10	2	0	0.20	128.3
AlphaBeta	pacman_maze	10	0	10	0.00	573.0
AlphaBeta	desperate_run	10	6	0	0.60	566.0
Minimax	tiny_hunt	10	10	0	1.00	490.2
Minimax	small_hunt	10	10	0	1.00	974.8
Minimax	open_field	10	10	0	1.00	1468.8
Minimax	drone_escape	10	10	0	1.00	993.0
Minimax	supply_run	10	10	0	1.00	993.0
Minimax	swamp_run	10	10	0	1.00	1450.7
Minimax	jungle_chase	10	7	3	0.70	483.0
Minimax	contested_territory	10	10	0	1.00	986.6
Minimax	maze_hunt	10	6	4	0.60	313.5
Minimax	mountain_pass	10	9	1	0.90	879.5
Minimax	wide_open	10	10	0	1.00	1480.8
Minimax	bottleneck	10	10	0	1.00	987.9
Minimax	gauntlet	10	2	8	0.20	-617.1
Minimax	triple_threat	10	7	3	0.70	1497.5
Minimax	scattered_hunters	10	5	5	0.50	658.8
Minimax	terrain_trap	10	8	2	0.80	748.9
Minimax	storm_tunnel	10	10	0	1.00	934.4
Minimax	arena	10	5	5	0.50	839.3
Minimax	hunter_cage	10	10	0	1.00	979.4
Minimax	tunnel_escape	10	10	0	1.00	487.0
Minimax	mountain_overfly	10	10	0	1.00	959.8
Minimax	hunter_swarm	10	2	8	0.20	231.8
Minimax	pacman_maze	10	0	10	0.00	104.8
Minimax	desperate_run	10	7	3	0.70	680.1

Al comparar las tasas de victoria de los tres agentes en los diferentes layouts se observa que, en general, los tres algoritmos obtienen un desempeño alto en mapas abiertos o menos restrictivos como ***tiny_hunt*, *open_field*, *supply_run* y *wide_open*,** donde alcanzan tasas de victoria cercanas al 100%. Sin embargo, en mapas más complejos o con mayor presión de los cazadores, como gauntlet, pacman***_maze y hunter_swarm,*** el rendimiento disminuye significativamente para todos los agentes. En particular, Expectimax tiende a mantener tasas de victoria ligeramente más estables en algunos mapas debido a que modela el comportamiento de los cazadores como probabilístico, mientras que Minimax y AlphaBeta asumen un adversario completamente racional. Además, se observa que AlphaBeta logra resultados similares a Minimax, lo cual es consistente con la teoría, ya que ambos algoritmos realizan la misma búsqueda, pero Alpha-Beta reduce el número de nodos evaluados mediante poda. En general, los resultados muestran que la estructura del layout tiene un impacto importante en el desempeño del agente, especialmente en mapas con múltiples amenazas o espacios reducidos.

- **¿Cuántos nodos adicionales poda Alfa-Beta en los mejores y peores casos?**

Se espera que en su análisis compare cualitativamente el desempeño de los distintos algoritmos en los layouts que encuentre de interés, así como que compare cuantitativamente métricas de complejidad computacional, tasa de victorias y tiempo de ejecución.

El algoritmo Alpha-Beta permite podar nodos adicionales del árbol de búsqueda en comparación con Minimax al evitar explorar ramas que no pueden afectar la decisión final. En el mejor caso, cuando los nodos se ordenan de manera favorable, Alpha-Beta puede reducir significativamente el número de nodos evaluados, llegando incluso a disminuir la complejidad efectiva de $O(b^d)$ a aproximadamente $O\left(b^{\frac{d}{2}}\right)$. Esto significa que el agente puede explorar arboles más profundos en el mismo tiempo de ejecución. En contraste, en el peor caso, cuando el orden de exploración es desfavorable, Alpha-Beta no logra realizar poda y termina evaluando prácticamente el mismo número de nodos que Minimax. En los experimentos realizados se observa que ambos algoritmos obtienen resultados de victoria similares en la mayoría de los layouts, lo cual es consistente con la teoría, ya que Alpha-Beta no cambia la decisión final del algoritmo, sino que únicamente optimiza el proceso de búsqueda. Sin embargo, la principal ventaja de Alpha-Beta se refleja en la reducción del tiempo de ejecución y en la capacidad de escalar mejor a arboles de búsqueda más grandes.

c) Reflexión uso de IA generativa

Durante el desarrollo de la solución se utilizó IA como apoyo para revisar errores en la implementación de los algoritmos de búsqueda adversaria y para estructurar el análisis experimental de los resultados. Una de las principales utilidades fue identificar posibles causas de comportamientos incorrectos del agente, como ciclos o decisiones subóptimas en Minimax, y sugerir mejoras en la estructura del código y en la forma de realizar las pruebas. También se utilizó para organizar los resultados obtenidos en múltiples layouts y facilitar su comparación cuantitativa.

Un aprendizaje importante fue el impacto que tenía la calidad de respuesta dependiendo del prompt que se le proporcionaba. En varias ocasiones fue necesario proporcionar más informaciones sobre el problema para obtener una solución realmente útil. Además, el proceso permitió comprender mejor el balance que existe en los algoritmos de búsqueda entre la profundidad de exploración de árbol, el costo computacional y la calidad de la función de evaluación. En particular, se observó que aumentar la profundidad mejora la calidad de las decisiones, pero incrementa significativamente el tiempo de ejecución, mientras que una buena heurística puede compensar parcialmente una profundidad menor.

En general, las propuestas sugeridas por la IA fueron comprensibles y sirvieron como guía para mejorar la implementación, pero siempre fue necesario revisarlas y adaptarlas al código del proyecto. Por esta razón, el uso de IA funciono principalmente como una herramienta de apoyo para depuración, analisis y organización del trabajo, sin reemplazar la comprensión de los algoritmos estudiados.